Variability-Aware Analysis of Kbuild Makefiles

ANONYMOUS AUTHOR(S)

The Linux kernel is highly configurable, and its `kbuild` Makefiles decide how thousands of configuration options affect which source files are included in the kernel build. A developer who asks why a source file is not built, or which options would build it, needs its build condition, a logical formula over configuration options. Running Make reveals the files of one configuration only, and enumerating all configurations is intractable. Symbolic execution that forks a state at every conditional suffers from path explosion, and prior static analyses of Kbuild Makefiles represent a variable as a set of complete string values. We present KFOLD, a symbolic execution of `kbuild` Makefiles that maps each Make variable to a set of words, each guarded by a path condition over configuration options, in the style of a value summary. Appends add guarded words, an overwrite re-guards the words it replaces, and path merging joins the two arms of a conditional into one symbolic state, so the state grows with the number of words instead of the number of paths. The resulting build conditions are SMT formulae that a solver can check and satisfy.

We evaluate KFOLD on Linux 7.2.8 and four other `kbuild`-based systems by comparing its build conditions under concrete configurations with the source files built by fresh builds. On four Linux configurations, from `tinyconfig` to `allmodconfig`, the predicted files agree with the built files, with no false positive and no miss, from 512 to 29,556 files. KFOLD analyzes the 2,215 Makefile instances of the Linux x86 tree in 143.1 s. The build conditions explain the build status of all 400 sampled Linux files and, combined with a Kconfig model, produce configurations for 384 of the 402 commits of a Linux stable update that touch C or assembly files.

CCS Concepts: • **Software and its engineering → Software verification and validation**.

Additional Key Words and Phrases: Kbuild, Makefile, symbolic execution, variability, Kconfig

1 Introduction

placeholder: add later [**?**]

2 Background

3 Motivating Example

This section follows one Kbuild Makefile through KFOLD to show what a build condition is and how the symbolic state stays small. Kbuild is the part of the Linux build system that uses Make to decide which source files are included in the kernel build, and Kconfig is the language that declares the configuration options and their dependencies. A Kconfig option is a tristate option when it takes the value `y` (built into the kernel), `m` (built as a loadable module), or unset. A Kbuild Makefile lists files in goal definitions: `obj-y` names the files built into the kernel and `obj-m` names the files built as modules. A line such as `obj-$(CONFIG_EXT4_FS) += ext4.o` computes the name of the goal definition from the value of an option.

?? is adapted from the Makefile of the Linux `ext4` file system. A developer who changes a file of this directory and sees a successful build may ask why `ext4.o` was not compiled, and which options would compile it. Running Make answers this question for the one configuration at hand. A build condition answers it for every configuration: a logical formula over configuration options that states when the file is built.

Let E_y and E_m stand for `CONFIG_EXT4_FS=y` and `=m`, T_y and T_m for the same values of `CONFIG_EXT4_KUNIT_TESTS`, and V for `CONFIG_FS_VERITY=y`. KFOLD executes the Makefile symbolically and keeps each Make variable as a set of words, each guarded by a path condition over these options, so that the membership of each word is a value summary in the sense of MultiSE [**?**]. Line 1 adds `ext4.o` to `obj-y` under E_y and to `obj-m` under E_m. Line 2 is an overwrite that applies under only some options: under T_y it replaces `obj-y` with `ext4-inode-test.o`, and under T_m it replaces `obj-m`. KFOLD therefore adds `ext4-inode-test.o` to `obj-y` under T_y and to `obj-m` under

```
1 obj-$(CONFIG_EXT4_FS) += ext4.o
2 obj-$(CONFIG_EXT4_KUNIT_TESTS) := ext4-inode-test.o
3 ifeq ($(CONFIG_FS_VERITY),y)
4   EXTRA := verity
5 else
6   EXTRA := crypto
7 endif
8 obj-y += $(EXTRA).o
9 ext4-inode-test-y += inode-test.o
```

Fig. 1. Makefile adapted from Linux `fs/ext4/Makefile`. Only line 1 is verbatim; lines 2 and 9 are adapted and lines 3–8 are constructed to add an overwrite, a conditional, and a composite object.

Table 1. Build conditions of the files built by **??**.

File	Built into the kernel (`obj-y`)	Built as a module (`obj-m`)
`ext4.o`	$E_y \wedge \neg T_y$	$E_m \wedge \neg T_m$
`ext4-inode-test.o`	T_y	T_m
`inode-test.o`	T_y	T_m
`verity.o`	V	—
`crypto.o`	$\neg V$	—

T_m, and keeps `ext4.o` only where the overwrite does not apply, which is $E_y \wedge \neg T_y$ for `obj-y` and $E_m \wedge \neg T_m$ for `obj-m`. Lines 3–7 assign `EXTRA` in both arms of a conditional; path merging joins the two arms into one symbolic state in which `EXTRA` is `verity` under V and `crypto` under $\neg V$. Line 8 then expands into two words with complementary conditions, and line 9 makes `inode-test.o` a member of the composite object `ext4-inode-test.o`, so it is built whenever its parent is. **??** lists the resulting build conditions. They are local to this Makefile: the condition of a file also includes the condition under which Kbuild descends into the directory `fs/ext4/`.

The build conditions answer the developer's question directly. Evaluated on the configuration at hand, they explain why `ext4.o` is absent, for example because T_y holds. Combined with the constraints of Kconfig, they are satisfiability modulo theories (SMT) formulae from which a solver finds a configuration that builds `ext4.o` by satisfying $E_y \wedge \neg T_y$. KFOLD offers these two uses as the commands `why` and `config-for`. A satisfying assignment still needs to be checked against Kconfig and a real build, which **??** reports.

The symbolic execution of Kbuild Makefiles by Nguyen and Nguyen [**?**] forks a symbolic state at every conditional and reduces the number of paths by path merging and by split-merge. In this example, the two tristate options and the conditional give $3 \times 3 \times 2 = 18$ paths before merging, and every further independent conditional can double the number. The two goal definitions above hold six guarded words in total, and each further append adds one more. MultiSE [**?**] introduced value summaries to avoid this growth in paths, and Kmax [**?**] also localizes configuration explosion to the statement level.

In short, guarded words per variable and path merging at each conditional give one build condition per file at a cost that grows with the number of words, so the conditions can be used for the questions that developers ask about a build.

4 Approach
5 Implementation
6 Evaluation
6.1 Experimental Setup
6.2 RQ1
6.3 RQ2
7 Related Work
8 Conclusion

Data-Availability Statement

Link to repository removed for double-blind review.

AI Use

placeholder: add later (counts toward the 20-page limit)

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.